

Software Architecture and Design Specification

Project: Task Manager Application

Version: 1.0

Authors: Team 4 — OM SATYAJYOTI PATTANAYAK; PRACHI GANESH JOSHI; PRIYANSH SONI; RAJAT KIRAN KOOGGE

Date: 02-10-2026

Status: Draft

Revision History

Version | Date | Author | Change Summary

1.0 | 02-10-2026 | Team 4 | Initial architecture and design specification

Approvals

Role | Name | Signature/Date

Team Members | Team 4 | Pending

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) defines the architecture and detailed design of Version 1.0 of the Task Manager Application. It translates the requirements baseline in the SRS into an implementable architecture, component responsibilities, interfaces, security controls, interaction flows, API design, error handling, and user-experience decisions. The document provides the design baseline for implementation and testing and maintains traceability to FR-01 through FR-11, NFR-01 through NFR-06, and SR-01 through SR-04.

1.2 Scope

The SAD covers the complete approved Task Manager Application scope:

- User registration and authentication.
- Creation, viewing, editing, and deletion of authorized tasks.
- Assignment of tasks to registered team members.
- Task status, priority, and deadline management.
- Search and filtering of authorized tasks.
- Dashboard-based task and progress information.
- Authentication, authorization, input validation, protected API operations, and persistent user/task data.
- React.js presentation, Node.js/Express.js application services, REST-style APIs, and persistent database storage.

Features outside the SRS, including notifications, advanced collaboration, production cloud administration, and other unapproved enhancements, remain outside the current design scope.

1.3 Audience

This document is intended for:

- Project team members and developers who implement the approved architecture and design.
- QA engineers who use the design for integration, system, security, and traceability testing.
- Project instructors and evaluators who verify alignment with the SRS and Part-1 deliverables.
- Security reviewers who examine authentication, authorization, input validation, password protection, and secure communication.
- Future maintenance teams who need to understand component responsibilities, interfaces, and design boundaries.

1.4 Definitions

SRS (Software Requirements Specification), SAD (Software Architecture and Design Specification), STP (Software Test Plan), RTM (Requirements Traceability Matrix), API (Application Programming Interface),

JWT (JSON Web Token), REST (Representational State Transfer), FR (Functional Requirement), NFR (Non-Functional Requirement), SR (Security Requirement), CRUD (Create, Read, Update, Delete).

2. Document Overview

2.1 How to use this document

This document is the architecture and design baseline for the Task Manager Application. It should be read together with the SRS and Test Plan. Section 3 describes the architecture, component responsibilities, architecture pattern, technology stack, risks, requirements traceability, and security architecture. Section 4 describes the design overview, UML sequence diagrams, API design, error handling, logging and monitoring, UX design, and approved future considerations. The design references the SRS requirement identifiers so that implementation and testing can be traced back to the approved requirements.

2.2 Related Documents

- Software Requirements Specification (SRS), Task Manager Application, Team 4, Version 1.0, 02-10-2026.
- Software Test Plan (STP), Task Manager Application, Team 4, Version 1.0, 02-10-2026.
- Requirements Traceability Matrix (RTM) contained in Appendix C of the SRS.

3. Architecture

3.1 Goals & Constraints

Goals:

- Provide secure, maintainable, reliable, usable task management with clear separation of concerns.
- Support all approved functional requirements FR-01 through FR-11.
- Provide measurable support for NFR-01 through NFR-06 and SR-01 through SR-04.
- Keep presentation, application/service logic, and data persistence responsibilities clearly separated.

Constraints:

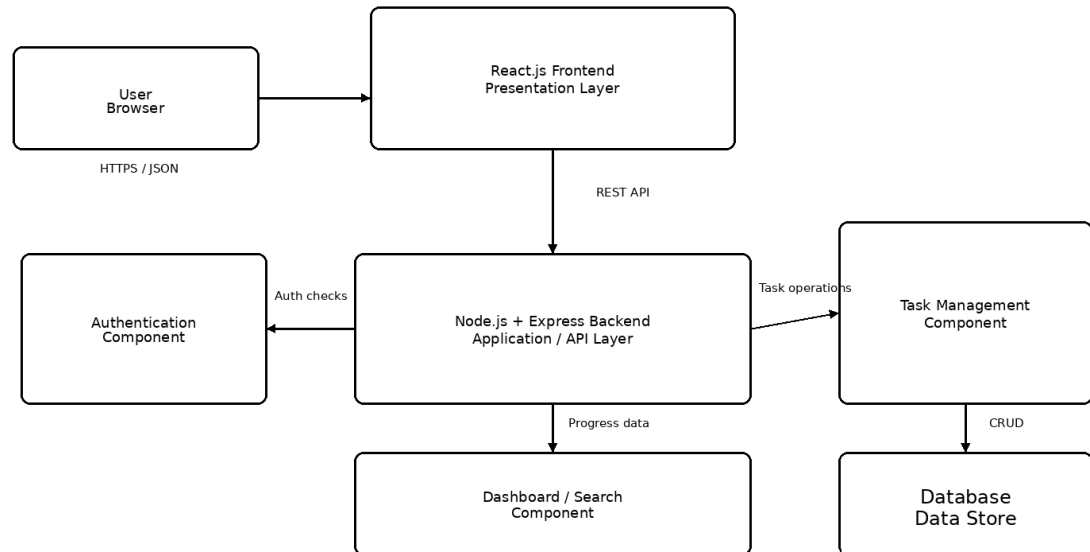
- React.js shall be used for the frontend.
- Node.js with Express.js shall be used for the backend.
- Persistent storage shall be provided for user and task information.
- Protected operations shall use authentication and authorization.
- Frontend/backend communication shall use defined APIs and structured request/response data.
- User input shall be validated before processing.
- Passwords and sensitive authentication information shall not be exposed in normal responses or logs.
- The design shall remain modular and independently testable.
- Implementation shall follow the approved SRS and SAD and shall not introduce unapproved functionality.

3.2 Stakeholders & Concerns

- Users: usability, clear task visibility, understandable navigation, and reliable task updates.
- Team members: assignment, status tracking, priority/deadline visibility, and access to authorized tasks.
- Task managers/task owners: task creation, editing, deletion, assignment, and progress management within authorization rules.
- Developers: modularity, maintainability, clear interfaces, and separation of responsibilities.
- QA engineers: testability, measurable behaviour, security validation, and requirements traceability.
- Project instructors/evaluators: conformance to the SRS, SAD, Test Plan, and required software-engineering deliverables.
- Security reviewers: authentication, authorization, input validation, password protection, and secure communication.

3.3 Component (UML) Diagram

Task Manager Application — UML Component Diagram



3.4 Component Descriptions

3.4 Component Descriptions

- **React Frontend:** provides the user interface for registration, login, task listing, task creation, task details, task editing, assignment, status/priority updates, search/filtering, and dashboard/progress views.
- **Authentication Module:** validates credentials, supports registration/login, establishes authenticated access, and supports protection of authenticated functions.
- **Task Management Module:** creates, retrieves, updates, assigns, and deletes tasks and manages task status, priority, deadline, and authorization-related task operations.
- **Dashboard / Search Module:** applies authorized task filtering/search and prepares task and progress information for presentation through the dashboard.
- **Node.js/Express API:** exposes REST-style endpoints, validates requests, applies authentication and authorization checks, returns appropriate HTTP/JSON responses, and coordinates backend services.
- **Database:** stores persistent user and task information, including task ownership/authorization relationships, assignment, status, priority, deadline, and other required task data.

3.5 Chosen Architecture Pattern and Rationale

3.5 Chosen Architecture Pattern and Rationale

A layered three-tier architecture is chosen to separate presentation, application/business logic, and data management. The React.js frontend forms the presentation layer; the Node.js/Express.js backend provides the application/service layer; and the database provides the persistence layer. This structure supports modular development, maintainability, testability, reliability, and clear responsibility boundaries required by the SRS.

A microservices architecture is not selected because the current approved project scope can be implemented as a modular single backend without introducing unnecessary distributed-system complexity. The layered structure also supports independent testing of major components and clear API boundaries.

3.6 Technology Stack & Data Stores

3.6 Technology Stack & Data Stores

- **Frontend:** React.js for the web-based user interface.
- **Backend:** Node.js with Express.js for application services and REST-style APIs.
- **Communication:** HTTP/HTTPS with JSON or another agreed structured request/response representation;

HTTPS shall be used for secure deployment.

- Data store: persistent database storage for registered users and task information, including the relationships required for assignment and authorization.
- Development and testing tools: modern web browser, Node.js tooling, Git, and API testing tools.
- The design does not require dedicated client hardware; the application runs on general-purpose computers using supported browsers.

3.7 Risks & Mitigations

3.7 Risks & Mitigations

- Risk: unauthorized access to protected task operations → Mitigation: require authentication and enforce authorization checks before protected API operations.
- Risk: invalid or malformed task/user data → Mitigation: server-side input validation and appropriate error responses.
- Risk: password or sensitive information exposure → Mitigation: secure password handling and exclusion of credentials from normal responses and logs.
- Risk: backend or database failure → Mitigation: standardized error handling, validation, and reliable persistence practices.
- Risk: changes in one module affecting unrelated modules → Mitigation: layered modular design and defined API interfaces.
- Risk: inconsistent task visibility → Mitigation: apply authorization rules to task retrieval, modification, assignment, search, filtering, and dashboard data.

3.8 Traceability to Requirements

3.8 Traceability to Requirements

The architecture and design are aligned with the Requirement Traceability Matrix in Appendix C of the SRS:

- FR-01 User registration → Authentication Module; API Design.
- FR-02 User authentication → Authentication Module; Login Sequence; API Design.
- FR-03 Create task → Task Management Module; Create Task Sequence; API Design.
- FR-04 View tasks → Task Management Module; API Design.
- FR-05 Edit task → Task Management Module; API Design.
- FR-06 Delete task → Task Management Module; API Design.
- FR-07 Assign task → Task Management Module; API Design.
- FR-08 Update task status → Task Management Module; API Design.
- FR-09 Task priority → Task Management Module; API Design.
- FR-10 Search and filter → Task Management Module / Dashboard Module; API Design.
- FR-11 Dashboard → Dashboard Module; UX Design.
- NFR-01 Performance → Architecture Goals & Constraints; Error Handling, Logging & Monitoring.
- NFR-02 Security → Security Architecture; Error Handling.
- NFR-03 Usability → Stakeholders & Concerns; UX Design.
- NFR-04 Reliability → Architecture Goals & Constraints; Error Handling, Logging & Monitoring.
- NFR-05 Maintainability → Architecture Goals & Constraints; Component Architecture.
- NFR-06 Testability → Architecture Goals & Constraints; API Design; Error Handling.
- SR-01 Authentication → Security Architecture; Login Sequence; API Design.
- SR-02 Authorization → Security Architecture; API Design.
- SR-03 Password Protection → Security Architecture; Error Handling, Logging & Monitoring.
- SR-04 Input Validation → Security Architecture; API Design; Error Handling.

Code-file and system-test references remain Part-2 implementation/test references as specified by the SRS RTM.

3.9 Security Architecture

3.9 Security Architecture

The security architecture follows defense-in-depth principles and directly addresses SR-01 through SR-04 and NFR-02:

- Authentication is required before protected task-management operations.
- Authorization checks are applied before users access, modify, assign, or delete task resources for which they lack permission.
- API input is validated on the server before registration, authentication, task, search, assignment, and update requests are processed.
- Passwords are handled using a secure mechanism selected during implementation and are not exposed in application responses or normal logs.
- HTTPS is used for secure deployment so credentials and task information are protected in transit.
- Access to task data is restricted according to the authenticated user and authorized task relationships.
- Authentication and authorization failures return appropriate HTTP responses without exposing sensitive information.

4. Design

4.1 Design Overview

4.1 Design Overview

The Task Manager Application is designed using a three-tier layered structure. The React presentation layer provides the web interface and communicates with the Node.js/Express application layer through REST-style APIs. The application layer manages authentication, authorization, validation, task creation/viewing/editing/deletion, assignment, status and priority updates, search/filtering, and dashboard data access. The database layer provides persistent storage for registered users and task information. This separation supports the maintainability and testability requirements defined in the SRS.

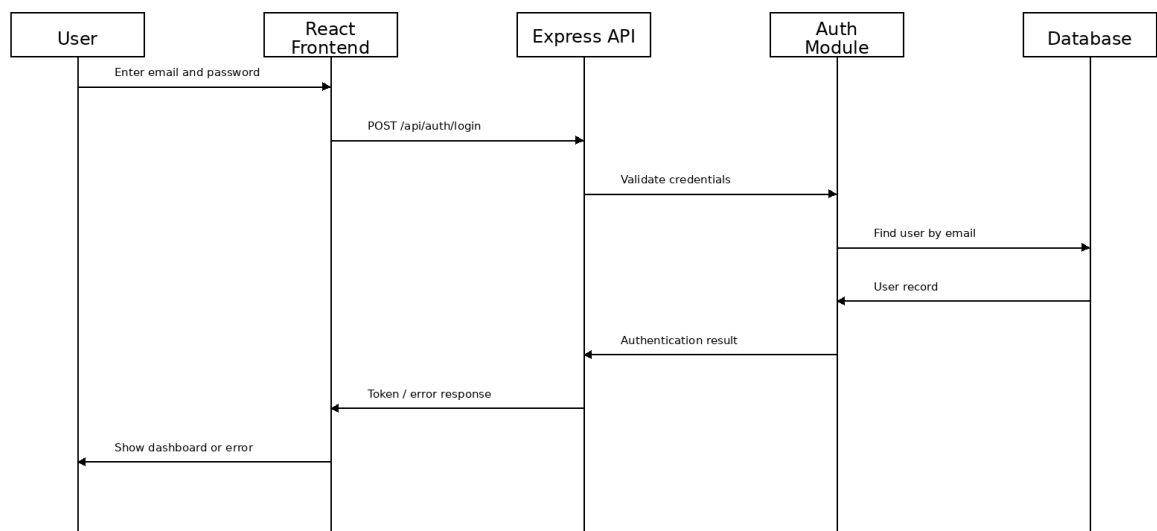
4.2 UML Sequence Diagrams

The following UML sequence diagrams cover two primary project flows:

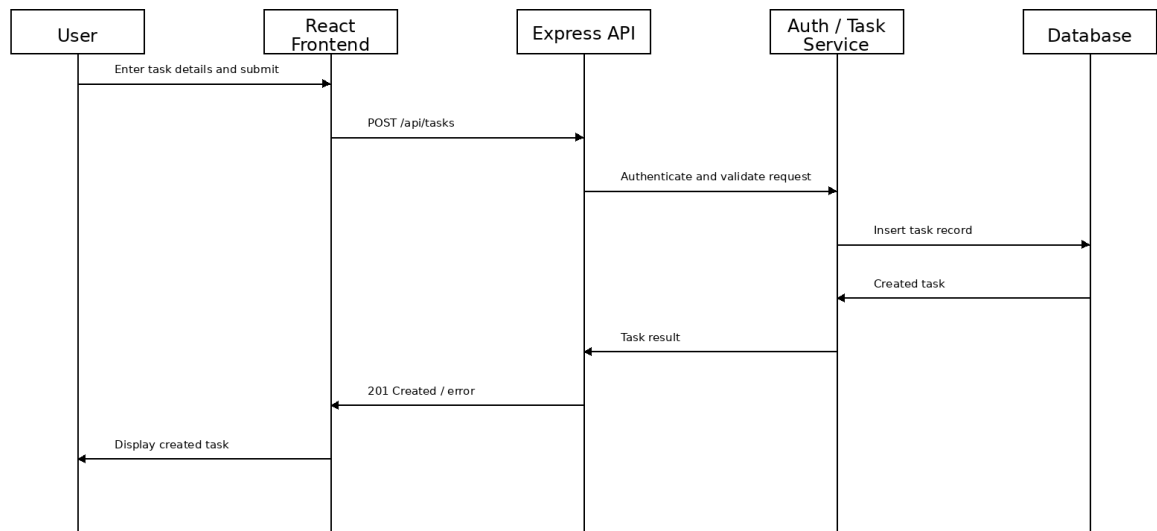
- User authentication: the user submits credentials through the React frontend; the Express API validates the request; the Authentication Module checks the user record; the result is returned to the frontend.
- Task creation: an authenticated user submits task details through the React frontend; the Express API authenticates and validates the request; the task service stores the task in the database; the created task/result is returned to the frontend.

These diagrams support FR-02 and FR-03 and the related security requirements identified in the SRS RTM.

Task Manager — UML Sequence Diagram: User Login



Task Manager — UML Sequence Diagram: Create Task



4.3 API Design

The API design defines the interfaces between the React frontend and the Node.js/Express backend. Authentication and task management are the primary API components. Protected task endpoints require authenticated access, and server-side validation and authorization are applied before task data is changed or returned.

Authentication API

POST /api/auth/register

Request: { email, password }

Response: { message, user }

POST /api/auth/login

Request: { email, password }

Response: { token, user }

Errors: 400 Invalid input, 401 Invalid credentials, 409 Email already registered.

Task API

GET /api/tasks

POST /api/tasks

PUT /api/tasks/:id

DELETE /api/tasks/:id

PUT /api/tasks/:id/status

PUT /api/tasks/:id/assign

Responses use JSON. Protected task endpoints require authenticated access. The API shall validate request data and authorization before performing protected operations.

Errors: 400 Invalid request, 401 Authentication required, 403 Unauthorized access, 404 Task not found, 500 Internal server error.

4.4 Error Handling, Logging & Monitoring

Standardized HTTP status codes and JSON error responses are used:

- 400 — invalid request or input.
- 401 — authentication required or invalid credentials.
- 403 — authenticated user is not authorized for the requested operation.
- 404 — requested task/resource is not found.
- 409 — duplicate email registration.
- 500 — internal server error.

Client-facing errors shall use clear messages without exposing sensitive implementation details. Authentication

credentials and other sensitive information shall not be written to normal application logs. Backend monitoring can track request failures, authentication failures, and server/database errors during deployment.

4.5 UX Design

The React interface shall provide a consistent, clear, and user-friendly experience across all major task-management activities. The interface shall provide dedicated and consistently structured forms and views for user registration, login, task creation, task viewing, task editing, task assignment, status and priority updates, search/filtering, and dashboard viewing. Navigation elements shall use understandable labels so that users can identify the purpose of each action without ambiguity.

The task creation and editing interfaces shall clearly present the required and optional task information, including task title, description, priority, deadline, status, and assignee where applicable. Required fields shall be clearly indicated, and users shall receive appropriate validation feedback when mandatory information is missing or invalid. The interface shall provide clear confirmation or error feedback after important operations such as creating, editing, assigning, updating, or deleting a task.

Validation and error messages shall be presented close to the relevant user action so that users can understand what went wrong and how to correct it. Authentication and authorization failures shall be communicated using clear messages without exposing sensitive implementation details. The interface shall also clearly communicate invalid operations or actions for which the current user does not have permission.

The task listing and task-detail views shall present information in a consistent manner so that users can easily identify the task, its current status, priority, deadline, and assignee where applicable. Search and filtering controls shall allow users to locate authorized tasks using supported task attributes. The dashboard shall provide clear progress indicators and summaries such as task counts, to-do tasks, in-progress tasks, completed tasks, and overdue tasks where applicable.

The UX design shall support the SRS usability requirement by maintaining consistent navigation, understandable labels, consistent presentation of task information, meaningful validation and error feedback, and clear progress indicators. At the same time, the interface shall restrict displayed task information to data that the currently authenticated user is authorized to access.

4.6 Open Issues & Next Steps

The current Version 1.0 implementation baseline focuses on the functionality defined in the approved SRS, including user registration and authentication, task creation and management, task assignment, status and priority management, search and filtering, dashboard-based progress tracking, security controls, and the associated functional and non-functional requirements.

Future enhancements may include notifications to inform users about relevant task events, assignments, status changes, or approaching deadlines. Advanced filtering and sorting may be introduced to provide more detailed ways of organizing and locating tasks. Richer analytics may provide additional information about task progress, completion patterns, workload, and other approved project metrics.

Further enhancements may also include role-based access improvements to provide more detailed permission levels, activity history to maintain a record of important task changes and actions, and additional collaboration features to support more advanced team interaction.

These enhancements are intentionally not included in the current Version 1.0 implementation baseline. Any such feature shall first be evaluated against the existing requirements, architecture, security considerations, and project scope. If approved, the corresponding requirements, architecture/design specifications, test cases, and requirements traceability information shall be updated through the project's requirements-change process before implementation.

The next steps for the current baseline are to proceed with implementation according to the approved SRS and SAD, maintain traceability between requirements and implementation, and subsequently validate the implemented functionality through the planned test cases and non-functional/security testing.

5. Appendices

5.1 Glossary: SRS, SAD, STP, RTM, API, JWT, REST, React.js, Node.js, Express.js, FR, NFR, SR, CRUD.

5.2 References: Software Requirements Specification for the Task Manager Application, Team 4, Version 1.0, 02-10-2026; Software Test Plan, Task Manager Application, Team 4, Version 1.0, 02-10-2026; Software Engineering Mini-Project Deliverables Part-1; supplied SAD template.

5.3 Tools: draw.io, PlantUML or equivalent UML tools, React.js, Node.js/Express.js, Git, API testing tools.